



北京大学
PEKING UNIVERSITY

Vibe Coding助力生物信息学

刘凤麟, 林乐天

- 定义: a software development practice assisted by artificial intelligence (AI) such as by chatbots. The software developer describes a project or task in a prompt to a large language model (LLM), which generates source code automatically. (Wikipedia)
- Vibe Coding将极大地加快个人创造的节奏
- 传统编程 vs. Vibe Coding:
 - 传统: 构思逻辑 -> 查阅文档 -> 编写语法 -> 手动调试。
 - Vibe: 描述意图 -> AI 生成代码并运行 -> **人类审查测试** -> 调整需求。



- **编码氛围**: 在知道如何设计代码架构、部署运维的前提下, 借助AI Coding的能力大幅缩短编码时间, 提高编码效率和代码质量。
- **应用氛围**: 只需提出应用的功能、目的、交互逻辑和使用细节, 仅需以极低的程度干预 (如资源采购或设计微调), 即可依托AI Agent的能力得到一款线上稳定运行的应用。
- **需求氛围**: 无须关心平台、部署运维、软件适配性、前后端逻辑、数据链路等细节信息, 只需提出需求交由AI实现。如果需求过于简单, 导致AI实现偏离预期, 也可以直接从需求描述层面纠偏, 直至完全实现软件类需求。

- Agent：能感知环境、自主规划并采取行动的 AI 系统。不仅能“说”，还能“做”。
- 底层机制解析：
 - MCP 协议 (Model Context Protocol)
 - 解决了“上下文”问题。
 - 让 AI 能够看懂你的本地文件树、读取终端的报错日志，知道你现在进行到了哪一步。
 - Function Call (函数调用) & Skills (技能)
 - 解决了“行动力”问题。
 - 大模型本身只输出文本，但通过 Function Call，它可以触发外部工具——比如自己运行 pip install，或者自己新建一个 .py 文件。这就是它的 Skills。



- 具体，而非抽象
 - 清晰思考
 - (×) 我想做一个待办事项应用
 - (√) 我需要一个可以按日期分类待办事项的应用，用户可以设置截止日期和优先级，完成的任务会被自动归档，还应该有一个每周进度统计功能.....
- 分步引导，而非一次性描述
 - 例如，在开发电子商务网站时，先专注于产品展示页面，确保设计和功能符合期望，再逐步添加购物车、用户账户和支付系统。
 - 渐进式开发更容易管理，也能在早期发现并纠正问题。
- 迭代，而非固执
 - 如果同一个问题询问了三四次都没能解决，重新开启一个对话，以新的逻辑提问。
 - 当上下文中累积了过多的错误信息时，AI的输出质量会进一步下降。

- 理解，而非执行
 - 一个有效的策略是让AI为代码添加详细的注释，明确说明自己是新手，需要简单易懂的解释。然后请AI讲解代码的运行原理。
 - 通过这种详细的解释，你往往能自己发现问题所在，结合注释，AI也能更快地协助你解决这些问题。
- 上下文意识与清晰引用
 - 明确引用先前讨论的内容，使用准确的术语和命名，避免模糊的代词和不完整的描述。
 - (×) 上面那个函数有问题
 - (√) login.js文件中的validateUser函数在处理空值时有问题

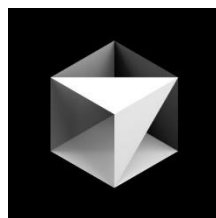
- C - Context (背景)：提供任务所需的上下文信息，包括目标受众、目的、限制条件等。
- R - Role (角色)：指定 AI 应扮演的角色（如“资深项目经理”“数据科学家”）。
- I - Instruction (指令)：明确告诉 AI 需要执行的具体任务。
- S - Subject (主题)：界定讨论的核心领域或话题。
- P - Persona (个性/风格)：设定输出的语言风格、语气或格式（如正式、幽默、Markdown）。
- E - Examples / Exception (示例/例外)：提供参考示例或明确排除不希望出现的内容。
- 示例整合提示词：
“作为一位环境科学家 (R)，请撰写一篇面向5岁儿童的科普文章 (C)，主题是气候变化 (S)。要求用通俗语言解释全球变暖 (I)，避免专业术语 (E)，风格轻松有趣 (P)。”

- 强大的模型
- 专业的AI编程环境
 - 智能代码提示与自动补全
 - 与多种语言模型的无缝集成
 - 有效的上下文管理，确保AI理解整个项目
 - 直观的代码编辑和调试工具
 - 便捷的版本控制集成
- 适合初学者的技术栈
 - 优先考虑拥有良好的文档、活跃的社区和成熟部署流程的技术
 - 前端：React（或Next.js）、Vue、Svelte
 - 后端：Node.js、Python（FastAPI或Django）
 - 数据库：PostgreSQL、MongoDB
 - 部署：Vercel、Netlify、Railway

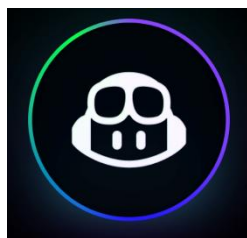
Web端 (canvas)



IDE/CLI like



cursor



Github
Copilot

Windsurf

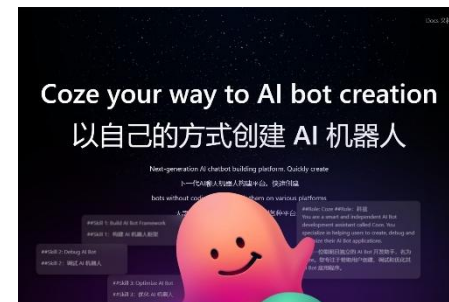


CLAUDE
CODE



TRAE

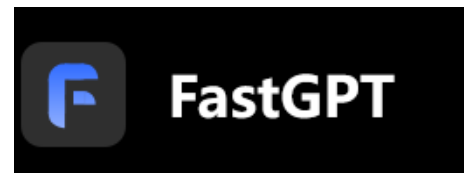
No-Code Agents & Workflows



Dify

Build Production Ready
Agentic Workflow

133.9K stars on GitHub



- 识别错误信息中的关键信息
- 分离和简化问题，创建最小复现场景
- 通过逐步排除法定位问题的根源
- 验证解决方案的有效性

- AI编程初期最常见的障碍
 - 例如，新项目初始化困难、编程语言之间的兼容性冲突、引入新库时的环境配置问题、不同库之间的依赖冲突、开发环境缺失或权限不足等
- 识别这类问题的特征（如安装错误、版本冲突、权限问题），并用清晰的方式向AI描述症状（如详细的错误信息和环境描述）。
- 处理策略：
 - 使用容器化技术（如Docker）创建一致的开发环境
 - 严格遵循项目推荐的环境设置流程
 - 使用版本管理工具（如nvm、pyenv）管理不同版本的语言环境
 - 记录完整的错误信息，包括控制台输出和日志

- 通常表现：程序行为与预期不符
- 解决关键：将复杂的情况分解为简单的步骤，逐一验证。在向AI描述问题时，清晰地说明预期行为和实际行为的差异，以及你对可能原因的猜测。
- 处理策略：
 - 将复杂的操作分解为更小的步骤，逐一验证
 - 使用打印语句或日志跟踪程序执行流程
 - 创建简化的测试场景，验证核心功能
 - 与AI一起审查关键代码逻辑，确保符合设计意图

- 与其让AI盲目地修复错误，不如让它帮助你理解代码的工作原理，使你能自己发现问题所在
- 解决关键：让AI为代码添加详细的注释，解释每一部分的功能和目的。
- 例如，要求AI：
 - 为复杂的函数添加行内注释，解释每一步的目的
 - 画出数据流图，展示信息如何在系统中流动
 - 解释算法的工作原理，包括边缘情况的处理
 - 提供可能的故障点列表，以及验证每个点的方法

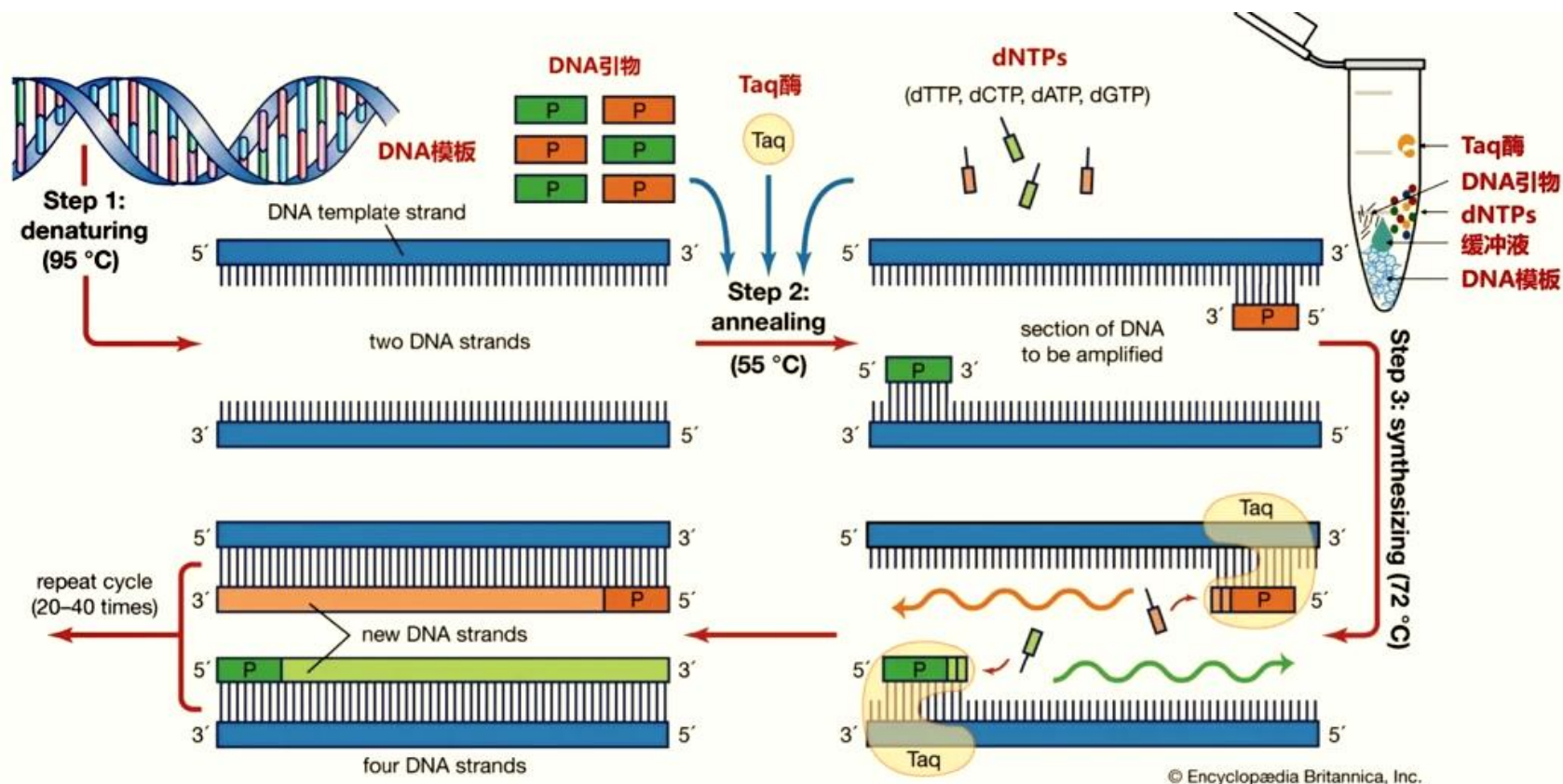
- 在面对复杂的问题时，创建最小化的复现环境，排除无关因素的干扰。
- 向AI描述这个简化后的问题，通常能得到更准确的解决方案。
- 保持系统简单，专注于核心功能。

- 先用AI/网络资源，后问人
- 深度探究
 - 当AI提供解决方案时，不要仅复制、粘贴，而应该要求AI解释背后的原理和逻辑。
 - 追问“为什么”，而不仅仅是“怎么做”
 - 要求AI提供多种解决方案，并比较它们的优缺点
 - 尝试预测方案可能遇到的问题，并与AI讨论这些问题
 - 要求AI解释代码中的设计决策和最佳实践
- 构建个人学习策略
 - 用户可以告诉AI自己的目标和现有知识水平，然后请它设计适合自己的学习步骤。
- 保持敬畏之心

- 面对复杂的项目，最有效的策略是问题拆解：将大问题分解为可管理的小问题。
- 识别系统的自然边界和组件，定义清晰的接口和交互方式，设计可独立实现和测试的模块，以及规划合理的实现顺序。
- 方向：
 - 功能拆分：将系统功能分解为相对独立的模块
 - 层级拆分：区分前端/后端，或按架构层次（如数据、业务逻辑、展示）拆分
 - 阶段拆分：将开发过程分为规划、核心功能、优化扩展等阶段
 - 复杂度拆分：优先解决关键问题，逐步增加复杂性。

- 今日目标：在不手敲一行 Python 逻辑代码的前提下，搭建一个带可视化界面的生物信息 Web 工具。
- 技术栈：Python + Streamlit (一种专为数据科学设计的极速 Web 框架)。
- 核心功能：
 - 接收 DNA 序列输入。
 - 计算 GC 含量 (GC Content) 与蛋白翻译 (Protein Translation)。
 - 根据专业参数自动设计 PCR 引物 (Primer Design)。

- 定义：引物（Primer）是一段短单链核酸（DNA或RNA），能与模板DNA特定区域互补结合，为DNA聚合酶提供起始合成3'-OH末端。
- 核心功能：在PCR、qPCR、测序、克隆等实验中启动DNA合成。



- 1. 长度与温度控制
 - 长度： 通常在 18-25 bp 之间。
 - 熔解温度： 理想范围在 55–65°C。上下游引物的 T_m 差距应小于 5°C。
- 2. 碱基组成
 - 比例： GC 含量应在 40% - 60%。
 - 分布： 尽可能均匀，避免出现连续 4 个以上的单一碱基或二核苷酸重复。
- 3. 3' 端 稳定性
 - 末端选择： 3' 端是酶延展的起点，最为关键。最好以 G 或 C 结尾
 - 避免冗余： 3' 端不应有超过 3 个连续的 G 或 C，否则会导致非特异性结合。
- 4. 避免二级结构
 - 发卡结构
 - 引物二聚体

TRAE的下载、安装、注册



许可协议

继续安装前请阅读下列重要信息。



请仔细阅读下列许可协议。您在继续安装前必须同意这些协议条款。

请您阅读以下协议条款，并在确认后勾选“我同意”继续下一步操作：

- TRAE 服务条款 (<https://www.trae.cn/terms-of-service>)
- TRAE 用户隐私协议 (<https://www.trae.cn/privacy-policy>)

☐ 我同意此协议(A)

☒ 我不同意此协议(D)

下一步(N)

取消



资源管理器

文件

打开的文件
夹里面的文件
都在这里

大纲

时间线

代码区

与 AI 对话

Ctrl + U

Editor 内 AI 编码

Ctrl + I

TRAE

AI对话区

与 Builder 协作

自动化执行常规开发任务，端到端高效完成，提高交付效率。

@Builder

@ #

MiniMax-M2.7

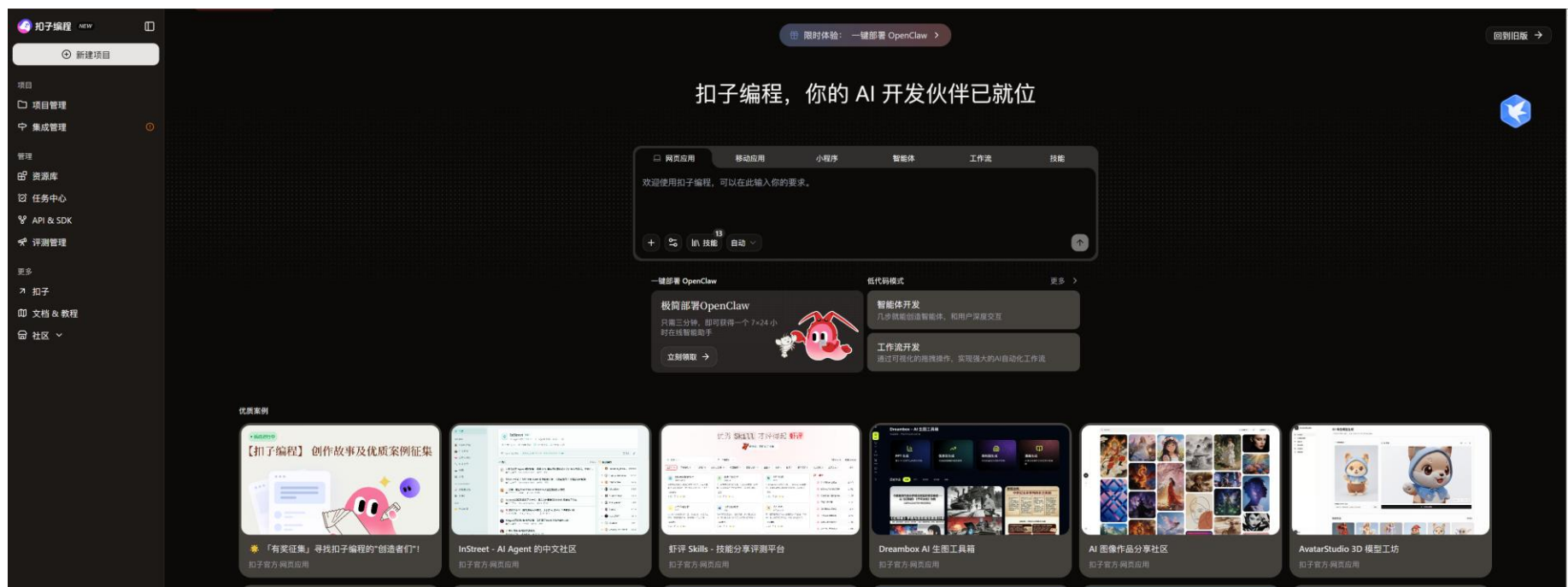
- Prompt: 用 Python 和 Streamlit 写一个 Web App。包含一个侧边栏。主页面放一个文本输入框, 让用户输入 DNA 序列。读取序列后, 在下方直接显示它的长度、GC 含量, 并翻译成蛋白质序列。
- 观察界面。观察 Builder 如何自动新建 app.py, 如何自动在终端运行。

- 尝试添加请求，让模型自动在现有代码基础上添加功能
- 添加引物预测功能
- Prompt：在页面上加一个 ‘设计引物’ 的按钮。点击后，帮我为刚才输入的序列设计一对引物。
- 现在设计出来的引物能用吗？

- 模糊的需求 = 垃圾的代码
- **重构 Prompt (采用 CRISPE 结构/精确规范):**
 - **任务:** 重写引物设计逻辑。绝对不要自己手写算法。你必须去调用业界最标准的 Python 库。
 - **约束条件 (Constraints):**
 - 引物长度: 18-22 bp。
 - Tm 值: 58-62 摄氏度。
 - GC 含量: 40%-60%。
 - 必须排查发夹结构。
- 最重要的是要说清楚你的需求。

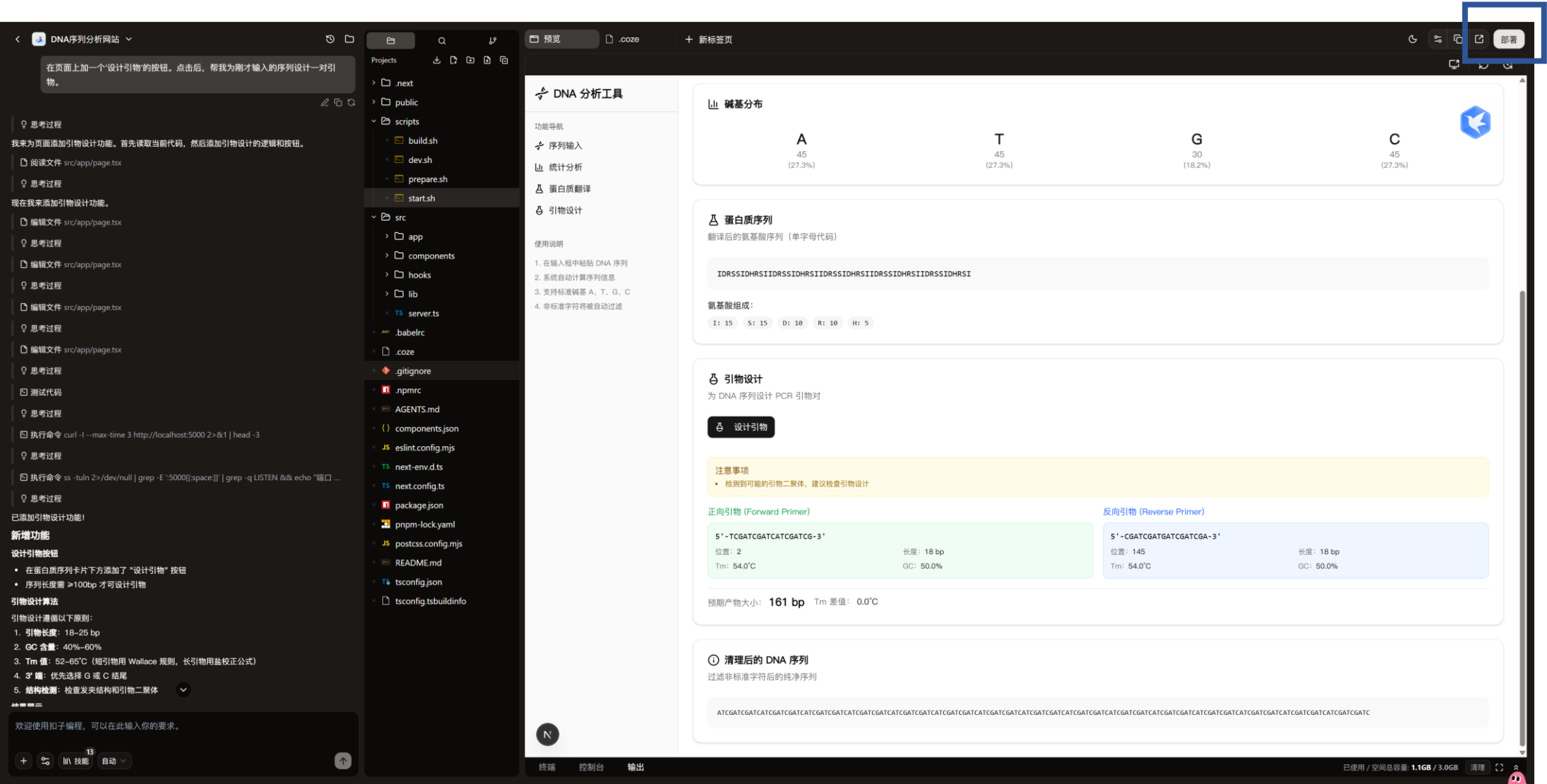
- 展示和做 UI 永远是开发的地狱，但恰好 AI 极其擅长做 UI
- Prompt:
 - 把找到的 Top 3 引物用 Streamlit 的 dataframe (数据表) 漂亮地展示出来，包含 Forward/Reverse 序列和 Tm 值。同时，在网页上把原始序列打印出来，并用红色高亮标出 Reverse 引物的结合位点。
- 由于随机性和模型能力差异等因素，需要多次调试，直到你对其感到满意。

- code.coze.cn
- 使用云端环境，一键部署



构建骨架





结合今天项目的体验，写出不少于 3 条在生物信息学编程中使用 vibe coding 的 tips 或心得体会，成功的经验或者失败的教训都可以。

• 提示：

1. 怎么写 prompt 才能让 AI 理解你的意思？
2. 怎么审查 AI 的输出？AI 写的命令可以运行吗？
3. 出现 bug 时，如何指导 AI debug？
4. 你觉得vibe coding和你传统写代码相比，有什么优势？有什么劣势？
5.

- 张昕东 《Vibe coding: AI编程时代的认知重构》
- 薛志荣, 池志炜 《零基础AI编程: Cursor助力Vibe coding实践》

Thank you!